

Trabajo Previo Laboratorio 09:

Introducción a las Listas y Manejo de Memoria en Python

Felipe Colli Olea

20 de mayo de 2026

1. Sobrecarga de Operadores: Listas Nativas vs Numpy

Al ejecutar las instrucciones `print(a+b)` y `print(a*2)` con listas nativas de Python, el intérprete no realiza operaciones aritméticas. En su lugar, el comportamiento definido por el lenguaje es la **concatenación** y **repetición** de secuencias. El resultado obtenido es:

- $a + b \rightarrow [1, 2, 3, 4, 5, 6]$
- $a * 2 \rightarrow [1, 2, 3, 1, 2, 3]$

Diferencia con arreglos de Numpy:

Si `a` y `b` fuesen estructuras del tipo `numpy.ndarray`, el comportamiento sería diametralmente opuesto debido a la vectorización. Numpy asume operaciones matemáticas *element-wise* (elemento a elemento) y *broadcasting* (transmisión de escalares). Los resultados hubiesen sido:

- $a + b \rightarrow [5, 7, 9]$ (Suma vectorial homóloga).
- $a * 2 \rightarrow [2, 4, 6]$ (Producto por un escalar).

2. Multiplicación de Secuencias

Al intentar ejecutar la instrucción `print(a*b)` con dos listas, se produce una interrupción de la ejecución, arrojando el siguiente error:

```
TypeError: can't multiply sequence by non-int of type 'list'
```

Razón: En Python, el operador de multiplicación (`*`) aplicado a listas está sobrecargado exclusivamente para aceptar un escalar de tipo entero (`int`) como segundo operando (indicando el número de veces que se repetirá la secuencia). El lenguaje no contempla una definición matemática para el producto entre dos secuencias estructuradas como listas (no calcula ni producto punto ni producto vectorial de forma nativa), por lo que el intérprete arroja una incompatibilidad de tipos (*TypeError*).

3. Mecánicas de Iteración y Mutabilidad

Se evaluaron dos bloques de código (*snippets*) que resuelven el mismo problema matemático: sumar y multiplicar los vectores `a` y `b` elemento a elemento. Ambos bloques imprimen exactamente los mismos vectores resultantes (`c = [4, 10, 18]` y `d = [5, 7, 9]`). Sin embargo, sus arquitecturas de manejo de memoria son distintas:

1. **Pre-asignación Estática** (`c[i] = ...`): El primer bloque crea listas inicializadas con ceros (`[0,0,0]`). Al recorrer el ciclo `for`, el programa accede a un espacio de memoria ya reservado y simplemente sobrescribe (*muta*) el valor contenido en ese índice. Es un acceso directo $O(1)$.
2. **Asignación Dinámica** (`c.append(...)`): El segundo bloque inicializa listas vacías (`[]`). En cada iteración del ciclo, el método `.append()` obliga a la estructura a expandir su tamaño dinámicamente en tiempo de ejecución para albergar el nuevo dato (comportamiento análogo a `std::vector::push_back()` en C++).

Imprimen lo mismo porque, lógicamente, el estado final de las estructuras de datos es idéntico, a pesar de que la ruta algorítmica para construirlas fue diferente.

4. Desafío Opcional: Ingreso de datos interactivo

A continuación, se presenta la implementación de la captura iterativa de valores por teclado para construir listas dinámicas:

```
a = []
b = []

print("Ingrese 3 valores para la lista 'a':")
for i in range(3):
    a.append(int(input(f"Valor {i+1}: ")))

print("Ingrese 3 valores para la lista 'b':")
for i in range(3):
    b.append(int(input(f"Valor {i+1}: ")))

print(f"Lista a resultante: {a}")
print(f"Lista b resultante: {b}")
```